A TECHNICAL WALKTHROUGH

RayTracerBench

Ray Tracing Theory, and How This Codebase Implements It

CPU vs. GPU (Metal compute) ray tracing benchmark · pure C++ and Metal

---

PART I

Theory

1. What a Ray Tracer Actually Computes

A ray tracer answers one question, over and over: *if I stand at the camera and look through this one pixel, what color do I see?* It answers it by firing a ray — a half-line with an origin and a direction — out from the camera through that pixel, and following it into the scene to see what it hits. Whatever it hits determines the color; if it hits nothing, it sees the sky.

That's the entire idea. Everything else in this document — intersection math, materials, sampling, gamma correction — exists to answer that one question correctly and to make the answer look photographic rather than flat.

2. Cameras, Rays, and the Image Plane

A ray is just an origin point and a direction vector: `P(t) = origin + t · direction`. As `t` increases, `P(t)` traces out every point along the ray. Given a camera position, an image plane in front of it, and a pixel's location on that plane, constructing the ray for that pixel is a matter of subtracting the camera position from the point on the plane in world space.

This project's camera (`CameraGPU` in `ShaderTypes.h`) follows the classic "Ray Tracing in One Weekend" model directly: instead of storing field-of-view and aspect ratio and recomputing the viewport every ray, it precomputes an `origin`, a `lowerLeftCorner`, and `horizontal`/`vertical` span vectors once per render. A ray for normalized image coordinates `(s, t)` (each in `[0,1]`) is then just:

$$direction = lowerLeftCorner + s \cdot horizontal + t \cdot vertical - origin$$

The camera also models a **thin lens**, not a pinhole: instead of every ray originating from a single point, rays originate from a random point on a small disk (the `lensRadius`), offset along the camera's own right/up basis vectors (`u`, `v`). Objects exactly at the focus distance stay sharp because rays through them from different points on the lens still converge on the same spot; objects nearer or farther blur, because they don't. That's where

camera depth-of-field comes from — it's a direct, physical consequence of a non-zero aperture, not a post-process blur.

3. Ray–Sphere and Ray–Pyramid Intersection

Spheres are the one primitive this renderer (and the book it's based on) uses, because the intersection test reduces to a single quadratic equation. A point `P` lies on a sphere with center `C` and radius `r` exactly when `(P − C)·(P − C) = r²`. Substituting the ray equation `P(t) = origin + t·direction` and expanding gives a quadratic in `t`:

$$a \cdot t^2 + b \cdot t + c = 0$$

where `a = direction·direction`, `b = 2·(origin−C)·direction`, and `c = (origin−C)·(origin−C) − r²`. Using the "half-b" substitution (`halfB = b/2`, which cancels the factor of 2 out of the quadratic formula and saves a few multiplications) gives the two roots:

$$t = (-halfB \pm \sqrt{(halfB^2 - a \cdot c)}) \,/\, a$$

A negative discriminant (`halfB² − a·c < 0`) means the ray misses the sphere entirely — no real roots. Otherwise there are up to two intersection points; the renderer wants the *closest one that's actually in front of the ray and within the valid t-range*, so it checks the smaller root first and falls back to the larger root only if the smaller one is out of range (behind the camera, or beyond a previously-found closer hit from some other sphere).

Once a hit point is found, its **surface normal** — the direction pointing straight out of the sphere at that point — is simply `(hitPoint − center) / radius` (already unit-length, since it's a displacement of exactly one radius). One subtlety matters for correct shading: a normal should point *toward* whichever side of the surface the ray came from, so materials can tell whether a ray is entering or exiting an object (this matters most for glass). The renderer checks the sign of `direction·outwardNormal` — negative means the ray is hitting the outside of the surface — and flips the stored normal if not.

Ray–Polyhedron Intersection: The Slab Method

A sphere reduces to one quadratic because its surface is a single smooth equation. A square pyramid doesn't have that luxury — it's five flat faces meeting at hard edges — but it's still a **convex** shape, and every convex polyhedron can be written as the intersection of

a handful of half-spaces, one per face: `dot(n_i, P) ≤ d_i`, where `n_i` is face `i`'s outward normal and `d_i` is its distance from the origin. A point is inside the polyhedron exactly when it satisfies *every* face's inequality simultaneously.

This project uses the **Kay–Kajiya slab method** to intersect a ray with that intersection-of-half-spaces directly, without ever building explicit triangles first — the same technique that makes axis-aligned bounding box tests fast (a box is just 6 such half-spaces, one pair per axis), generalized here from 6 planes to 5. Substituting the ray equation into a face's plane inequality and solving for `t` gives, for each face:

$$t = (d_i - dot(n_i,\ origin)) \,/\, dot(n_i,\ direction)$$

The sign of the denominator tells you which side of that constraint the ray is moving into: a negative denominator means the ray is *entering* that half-space as `t` increases, so its `t` is a candidate lower bound (`tNear`); a positive denominator means the ray is *exiting*, so its `t` is a candidate upper bound (`tFar`). Running all 5 faces through this and taking the largest `tNear` and smallest `tFar` gives the ray's entire span inside the polyhedron in one pass — no triangle list, no per-triangle loop. A hit exists exactly when `tNear ≤ tFar`, and the entry point's face — whichever face produced the winning `tNear` — supplies the surface normal directly, with no separate normal computation needed.

For a square pyramid specifically (base half-width `s`, height `h`, apex above the base center), the 5 face planes fall out of the shape's own symmetry in closed form rather than needing numerical fitting. The base is simply `y ≥ 0` (normal `(0,−1,0)`, distance 0). Each side face passes through the apex and two adjacent base corners; working out the plane through, e.g., the `+X` face's corners `(s,0,±s)` and the apex `(0,h,0)` gives `h·x + s·y = s·h` — normal `(h, s, 0)` at distance `s·h`. The other three side faces follow immediately by symmetry (negate `x`, or swap `x` / `z` for the pair facing along `z`).

A BOUNDARY THIS METHOD DOESN'T HANDLE

The slab method above only ever resolves the ray's *entry* face — exactly what a normal, opaque surface needs. It does not correctly handle a ray whose *origin already starts inside* the solid, which is precisely what happens partway through tracing a refracted ray inside glass. This project's pyramids are therefore restricted to lambertian and metal materials, never dielectric — an explicit, documented scope limit rather than an oversight (see §10).

4. Materials: Diffuse, Metal, Glass

When a ray hits a surface, the material decides what happens next: does the ray bounce, in what direction, and how much of its color survives the bounce (the *attenuation*)? This renderer implements the book's three classic materials.

Lambertian (matte/diffuse)

A perfectly diffuse surface scatters incoming light equally in every direction above the surface. The physically-motivated way to sample that is to pick a random point on a small sphere sitting just above the hit point, tangent to the surface, and scatter toward it — which produces a cosine-weighted distribution of bounce directions clustered around the normal, matching how matte surfaces actually behave. The scattered ray's direction is simply `normal + randomUnitVector()`, and the surface's albedo color is multiplied in as the attenuation.

Metal (reflective, with fuzz)

A mirror reflects a ray about the surface normal: `reflected = incoming - 2·(incoming·normal)·normal` — the same reflection law as light bouncing off a real mirror, or a ball bouncing off a wall. A perfect mirror is visually harsh, so this material adds a `fuzz` parameter: the reflected direction is perturbed by a small random offset scaled by `fuzz`, which is exactly a `Random unit vector, scaled down` technique that turns a perfect mirror into a brushed-metal look as `fuzz` increases from 0.

Dielectric (glass)

Glass is the most involved of the three, because light hitting a transparent surface can either *refract* (bend and pass through) or *reflect*, and which one happens depends on the angle of incidence. Refraction follows Snell's law; this renderer uses the standard vector form (splitting the refracted ray into components perpendicular and parallel to the normal):

$$refracted = (\eta_i/\eta_t)\cdot(incoming + cos\theta\cdot normal) \; - \; \sqrt{(1 - |\cdot|^2)}\cdot normal$$

Two more things make glass look convincing:

- **Total internal reflection**: at a steep enough angle, light passing from a dense medium (glass) into a less dense one (air) can't refract at all — Snell's law would require `sinθ`

`> 1`, which is impossible. When that happens, the ray must reflect instead, no choice involved.

- **Schlick's approximation**: even when refraction is physically possible, real glass partially reflects at grazing angles (this is why a window looks like a mirror when viewed edge-on). Schlick's approximation gives a cheap, visually-convincing probability that a given ray reflects instead of refracts, and the renderer draws a random number to decide which happens for each individual ray.

5. Monte Carlo Path Tracing

A single ray per pixel produces a hard, aliased, noiseless-but-wrong image: every pixel is either "the ray hit exactly this one point on exactly this one bounce path" or "it didn't." Two techniques turn that into a photograph-like image.

Anti-aliasing via multi-sampling

Instead of firing one ray dead-center through each pixel, the renderer fires many (`samplesPerPixel`), each jittered by a random sub-pixel offset, and averages their resulting colors. Edges that would otherwise be a hard staircase of pixels become a smooth gradient, because some of a given pixel's samples land just inside an object's silhouette and some land just outside it.

Recursion, made iterative

A ray that hits a diffuse or metal or glass surface doesn't stop there — it scatters, and the new ray needs to be traced too, recursively, until it either hits nothing (the sky) or gets absorbed. The book's own code expresses this as literal recursion: `ray_color(scattered, world, depth-1)`. Metal Shading Language kernels cannot recurse (no call stack of arbitrary depth is available on a GPU thread), so this project's shared `rayColor()` expresses the exact same logic as a bounded `for` loop instead — walking forward through bounces, multiplying the running color by each surface's attenuation, and stopping either when a ray hits nothing (return the accumulated color times the sky) or when `maxDepth` bounces are exhausted (return black — that path contributed no more light). This is a mechanical transformation, not an approximation: an iterative loop accumulating a running product is exactly equivalent to the recursive version, just without needing a call stack.

6. Gamma Correction

Displays don't map pixel values to brightness linearly — a pixel value of 0.5 doesn't look half as bright as 1.0, it looks noticeably dimmer than that, because monitors (following a decades-old CRT convention that stuck) apply their own roughly-squaring response curve. A renderer that computes physically-linear light and writes it to the screen uncorrected produces images that look too dark, especially in mid-tones. The fix used here — and in the book — is the simple, standard **gamma-2** correction: take the square root of each linear color component before writing it out, which is the cheap, close-enough approximation photographers and GPU programmers have used for exactly this purpose since long before this book was written.

7. Pseudo-Random Numbers on a GPU

A path tracer needs a fast source of random numbers — for anti-aliasing jitter, for diffuse scatter directions, for the lens-disk sample, for the reflect-vs-refract coin flip in glass — and it needs that source to work identically whether it's running as a single CPU thread or as one of millions of near-simultaneous GPU threads, with no shared state and no locking between them. There's no GPU equivalent of CUDA's `curand` library available in Metal Shading Language, so this project uses a tiny, well-known **pcg-hash** construction instead: a pure function that takes a 32-bit integer state and returns a new, well-mixed 32-bit integer, with no external library and no shared memory required. Divide the result by 2^{32} and it's a uniform float in `[0,1)`; hash it again and it's the next draw in the stream.

PART II

The Code

8. The Constraint That Shapes Everything

The single decision that shapes this codebase's architecture is this: **Metal Shading Language forbids virtual functions, RTTI, and dynamic allocation inside kernel code.** The book this project is based on — and the CUDA port it structurally follows — both lean on C++ virtual dispatch for their `hittable`/`material` class hierarchies (a sphere "is-a" hittable; a lambertian material "is-a" material; call `->scatter(...)` and let dynamic dispatch pick the right implementation). That pattern is simply not legal MSL.

So both renderers here — CPU *and* GPU — use the same alternative: flat arrays of plain structs, with a small integer **tag** (`MaterialType`) saying which kind of material a given entry is, and a `switch` statement dispatching on that tag instead of a virtual call. This isn't merely a GPU workaround bolted onto an otherwise object-oriented CPU renderer — both renderers run the identical tagged-struct code, which is what makes the CPU/GPU performance comparison this project exists to make actually meaningful. Two independently-written implementations that merely *look* similar wouldn't prove anything about CPU vs. GPU performance; running the same algorithm source on both does.

9. The Shared Core: `ShaderTypes.h` and `RayTraceCore.h`

`Core/ShaderTypes.h` defines the plain data — geometry, materials, the camera, and render parameters — as ordinary structs built from `simd_float3` (Apple's SIMD vector type, which Apple's headers make available, byte-for-byte identical, in both plain C++ and Metal Shading Language via `<simd/simd.h>`):

Core/ShaderTypes.h

```
enum MaterialType
{
        MAT_LAMBERTIAN = 0,
        MAT_METAL      = 1,
        MAT_DIELECTRIC = 2,
};

struct MaterialGPU
{
        int         type; // MaterialType
        simd_float3 albedo;
        float       fuzz; // metal roughness in [0,1]; unused for other types
        float       ir;   // dielectric index of refraction; unused for other types
};

enum ShapeType
{
        SHAPE_SPHERE  = 0,
        SHAPE_PYRAMID = 1,
};

struct TransformGPU
{
        simd_float3 position;
        simd_float3 right, up, forward; // orthonormal orientation basis
};

struct ShapeGPU
{
        int   type;          // ShapeType
        float radius;        // sphere: radius. pyramid: unused.
        float baseHalfWidth; // pyramid: half the base square's side length. sphere:
unused.
        float height;        // pyramid: apex height above the base. sphere: unused.
        int   materialIndex; // index into the parallel materials array
};
```

ECS-Style Geometry: Entities as Component Arrays

Once a second primitive (the square pyramid) entered the picture, a `SphereGPU`-only representation stopped making sense — geometry needed to be pluggable the same way materials already were. Rather than reach for a polymorphic shape hierarchy (illegal in MSL kernels for the same reason materials avoid it — see §8), this project organizes geometry the way an **Entity Component System** would: an *entity* is nothing but a plain array index; its *components* are the data at that index in a set of parallel arrays; and a *system*

is a function that walks those arrays and does something with them, dispatching on a tag rather than a virtual call.

Concretely: `TransformGPU` is the Transform component (a position plus an orthonormal orientation basis — meaningless for a sphere, which looks identical under any rotation, but essential for a pyramid's apex direction and base orientation), and `ShapeGPU` is the Shape component (a tagged union of per-primitive dimensions, plus a `materialIndex` that is itself a component reference into the materials array). `Core/Scene.hpp`'s `SceneDescription` stores these as parallel `transforms`/`shapes` arrays indexed by entity — see §10. Materials were already organized this way from the start (§8); geometry simply caught up to the same idea once it needed the same flexibility.

`Core/RayTraceCore.h` is where the theory from Part I actually lives in code — and it's `#include`d **verbatim** by both `Shaders/Raytracer.metal` (the GPU kernel) and `CPU/CPURenderer.cpp`. Not "a similar copy kept in sync by hand" — the literal same header, compiled twice, once by `clang++` and once by Apple's Metal shader compiler. Any divergence between what the CPU and GPU actually compute is impossible by construction, rather than merely a bug to watch out for.

The address-space problem, and how this header avoids it

Making one header compile as both plain C++ and Metal Shading Language runs into a real technical obstacle: MSL requires pointers to be tagged with an *address space* keyword (`device`, `thread`, `constant`, ...) saying where the pointed-to memory actually lives, and plain C++ has no equivalent concept at all. Get this wrong and the Metal compiler rejects the shader outright.

This header sidesteps the problem almost entirely by a single design rule: **every function passes and returns plain values, never references or pointers into caller-owned storage** — with exactly one deliberate exception. Hit outcomes, scatter results, and RNG state all travel as small return-value structs (`HitResult`, `ScatterResult`, `RandomFloatSample`, ...) instead of the more conventional C-style out-parameters. The one place a real address-space-tagged pointer is unavoidable is `rayColor()`'s scene array parameters — the actual entity-component and material buffers a GPU kernel receives — and that's handled by a single macro:

Core/RayTraceCore.h

```
#if defined( __METAL_VERSION__ )
        #define RT_DEVICE device
#else
        #define RT_DEVICE
#endif

inline RayColorResult rayColor( Ray r,
        RT_DEVICE const TransformGPU* transforms, RT_DEVICE const ShapeGPU* shapes,
uint32_t entityCount,
        RT_DEVICE const MaterialGPU* materials, uint32_t maxDepth, uint32_t rngSeed )
{ /* ... */ }
```

Everywhere else in the call chain — `hitSphere()`, `hitPyramid()`, `scatter()` — a single entity's components or material are taken *by value*, read out of that device-space array exactly once at the call site (`transforms[i]`, `shapes[i]`) and copied into ordinary local variables before being passed anywhere further. MSL treats those copies as plain thread-local data needing no annotation at all, so the address-space concern never propagates past `rayColor()` itself. One macro, one place it's used — not a general-purpose shimming mechanism sprinkled everywhere.

`hitEntity()`: The Collision System

This is the direct code equivalent of §9's ECS description above — the "system" that walks the Transform and Shape component arrays and performs the ray/geometry intersection, dispatching on `ShapeGPU::type` instead of a virtual call, exactly like `scatter()` already dispatches on `MaterialGPU::type`:

Core/RayTraceCore.h (abridged)

```
inline HitResult hitEntity( TransformGPU transform, ShapeGPU shape, Ray r, float tMin,
float tMax )
{
        switch ( shape.type )
        {
                case SHAPE_SPHERE:  return hitSphere( transform, shape, r, tMin, tMax
);
                case SHAPE_PYRAMID: return hitPyramid( transform, shape, r, tMin, tMax
);
        }
}
```

`hitPyramid()` is where §3's slab-method theory becomes code. It first transforms the incoming world-space ray into the pyramid's own local space — not via a matrix inverse, but by projecting the ray's origin and direction onto the Transform's `right` / `up` / `forward`

basis vectors directly (projecting onto an orthonormal basis *is* the inverse rotation, since an orthogonal matrix's inverse is just its transpose). The local-space intersection (`hitPyramidLocal()`) then runs the 5-plane slab test from §3 exactly as derived there, and the resulting hit normal is mapped back to world space by running the same basis forward. Because translation and rotation are both distance-preserving, the ray's `t` parameter is identical in local and world space — only the normal needs transforming back; the world-space hit point is simply `rayAt(r, t)`, computed directly.

The same "shared header, two compilers" idea produced one more small but real portability lesson, worth calling out because it's the kind of thing that's easy to get wrong quietly: `simd_float3(x, y, z)` function-call-style construction compiles fine in Metal Shading Language, but plain C++ rejects it (a C++ compiler parses it as an invalid functional cast with too many arguments — brace-initialization or `simd_make_float3(x,y,z)` are needed instead). `RayTraceCore.h` works around this with its own tiny `makeFloat3()` helper, implemented once per side of the `#ifdef`, so every other line of the shared algorithm can just call `makeFloat3(x, y, z)` without caring which compiler is currently reading it.

Scatter, as a tagged switch

This is the direct code equivalent of Part I §4 — reflection, refraction, Schlick's approximation, and the Lambertian random-scatter-direction, all dispatched by a plain `switch` on `mat.type` instead of a virtual call:

Core/RayTraceCore.h (abridged)

```
inline ScatterResult scatter( Ray rayIn, HitRecord rec, MaterialGPU mat, uint32_t
rngSeed )
{
        switch ( mat.type )
        {
                case MAT_LAMBERTIAN:
                {
                        RandomVec3Sample rv = randomUnitVector( rngSeed );
                        simd_float3      scatterDirection = rec.normal + rv.value;
                        if ( nearZero3( scatterDirection ) ) scatterDirection =
rec.normal;
                        return ScatterResult{ true, mat.albedo, Ray{ rec.p,
scatterDirection }, rv.seed };
                }
                case MAT_METAL:
                {
                        RandomVec3Sample rv = randomInUnitSphere( rngSeed );
                        simd_float3      reflected = reflect3( normalize3(
rayIn.direction ), rec.normal );
                        Ray              scattered = Ray{ rec.p, reflected + mat.fuzz *
rv.value };
                        bool             ok = dot3( scattered.direction, rec.normal ) >
0.0f;
                        return ScatterResult{ ok, mat.albedo, scattered, rv.seed };
                }
                case MAT_DIELECTRIC:
                {
                        /* Snell's law + total-internal-reflection check + Schlick's
approximation,
                           exactly as derived in Part I §4 */
                }
        }
}
```

FALLBACK, DOCUMENTED UP FRONT

If sharing this header verbatim ever proved too fussy in practice, the project's own design notes call out a fallback: hand-duplicate the core functions into a CPU copy and a GPU copy, marked with matching `// KEEP IN SYNC` comments at both ends, and treat any future divergence between them as a bug. In practice this fallback was never needed — the shared-header approach worked and was validated directly against the real Metal shader compiler (see §14) before any renderer code was built on top of it.

10. Building the Scene

`Core/Scene.hpp/.cpp` builds the classic "Ray Tracing in One Weekend" demo scene once — a large ground sphere, a randomized field of small spheres (diffuse, metal, and glass in roughly an 80/15/5 split, following the book's own proportions), three large "feature" spheres (one of each material), and a handful of square pyramids (below) — from a seeded `std::mt19937`, into a single `SceneDescription` that *both* renderers consume unmodified:

Core/Scene.hpp

```
struct SceneDescription
{
        std::vector<TransformGPU> transforms;
        std::vector<ShapeGPU>     shapes;
        std::vector<MaterialGPU>  materials;
        CameraGPU                 camera;
        RenderParams               params;
};

SceneDescription buildDefaultScene( unsigned seed, uint32_t width, float aspectRatio,
        uint32_t samplesPerPixel, uint32_t maxDepth, bool floating = false );
```

Because `Scene.cpp` is pure host-side C++ — it never gets compiled as a Metal shader — it's free to use the full `simd_` C API directly (`simd_make_float3`, `simd_normalize`, `simd_cross`) rather than the hand-rolled portable helpers `RayTraceCore.h` needs. There is no separate "GPU version" of the scene at all: the exact same seed produces the exact same entity layout and materials on both renderers, which is precisely what makes the deterministic-parity testing in §14 meaningful in the first place. Every entity is spawned through one small helper, `addEntity()`, which pushes a `TransformGPU` / `ShapeGPU` pair onto the two parallel arrays at the same index — spheres and pyramids are interleaved in the same two arrays, distinguished only by `ShapeGPU::type`.

Square Pyramids, at Varied Orientations

Five pyramids sit in the demo scene at varied yaw/tilt orientations — deliberately the shape that exercises `TransformGPU`'s orientation basis at all, since (as §9 notes) a sphere looks identical under any rotation. Each pyramid's orientation is built from two independent rotations — a tilt around the local Z axis, then a yaw around world Y — so some pyramids are merely spun in place while others are visibly tipped over.

Placing a tilted pyramid flush on the ground without it clipping through or floating above it turned out to have an exact, closed-form answer: transform the pyramid's 5 local vertices (apex plus 4 base corners) through its orientation basis, find whichever one lands lowest, and shift the whole pyramid up or down so that vertex sits exactly at ground level. Unlike `kMaxFloatHeight` below (which needed a rendered check against the camera frustum because no trusted closed form was found), this *is* solvable exactly — so every pyramid, at any tilt, rests flush on the ground with no per-orientation eyeball check required.

Pyramids are always lambertian or metal, never dielectric — see §3's note on why the slab intersection method doesn't support a ray originating inside the solid, which glass refraction requires once a ray exits the far side.

The "Floating?" checkbox

The `floating` parameter (default `false`, so every existing caller and test is unaffected) is the scene-side half of the UI's "Floating?" checkbox (see §13). When `true`, each small randomized-field sphere's height is drawn uniformly from `[radius, kMaxFloatHeight]` instead of resting on the ground, via one small helper:

Core/Scene.cpp

```
float placementHeight( bool floating, float radius, float restingHeight,
        std::mt19937& rng, std::uniform_real_distribution<float>& unit )
{
        if ( !floating ) return restingHeight;
        return radius + unit( rng ) * ( kMaxFloatHeight - radius );
}
```

The three large feature spheres (glass, lambertian, metal) always call this with `floating = false` regardless of the checkbox — an explicit design choice made after an initial version floated every sphere, feature spheres included, and looked worse for it. `kMaxFloatHeight` itself (3.0 world units) wasn't derived from the camera's field of view analytically — an exact per-object bound was worked out by hand from the camera's `u` / `v` / `w` basis vectors, but a hand-checked example didn't verify cleanly, so it was dropped in favor of the simpler, directly-checkable approach used throughout this project: render the fixed camera setup at candidate heights and confirm by eye that nothing leaves the frame, on both renderers.

11. The CPU Renderer

`CPU/CPURenderer.hpp/.cpp` is deliberately plain C++17 with zero Apple-framework dependencies. For each pixel and each sample, it calls the exact same `getRay()` / `rayColor()` from the shared core, accumulates the resulting color, applies the gamma-2 correction from Part I §6, and writes an RGBA8 byte quad:

```
uint8_t toByte( float c )
{
        float gammaCorrected = std::sqrt( std::max( 0.0f, c ) );
        float clamped = std::min( gammaCorrected, 0.999f );
        return static_cast<uint8_t>( 256.0f * clamped );
}
```

Threading is plain `std::thread` row-partitioning (each thread renders a contiguous band of image rows) — not Grand Central Dispatch, so that `Core/` and `CPU/` together stay entirely framework-free. A `CPUThreading::SingleThreaded` / `MultiThreaded` toggle is exposed explicitly, defaulting to multi-threaded (a realistic CPU baseline), specifically so a displayed speedup number is never ambiguous about which CPU configuration it was measured against.

Each pixel derives its own RNG seed from the scene's shared `frameSeed` hashed together with that pixel's coordinates, so every pixel gets an independent random stream — important both for correctness (adjacent pixels shouldn't be correlated) and for a property that turned out to matter for testing: rendering the same scene single-threaded and multi-threaded produces *byte-identical* output, since each pixel's result depends only on its own coordinates and the shared seed, never on which thread happened to compute it or in what order.

12. The GPU Renderer

`GPU/GPURenderer.hpp/.cpp` is written directly against Apple's `metal-cpp` C++ bindings for the Metal API — no MetalKit, no Objective-C. The device, command queue, and compute pipeline state are created once, in the constructor, and cached for the renderer's whole lifetime, so that Metal's shader-compile latency (which can be tens of milliseconds) never contaminates a timed render.

`Shaders/Raytracer.metal` is the GPU counterpart to `CPURenderer.cpp`: one compute kernel, one thread per pixel, calling the exact same shared `getRay()` and `rayColor()`:

Shaders/Raytracer.metal (abridged)

```
#include "../Core/ShaderTypes.h"
#include "../Core/RayTraceCore.h"

kernel void renderKernel(
        device const TransformGPU* transforms [[buffer( 0 )]],
        device const ShapeGPU*     shapes [[buffer( 1 )]],
        constant uint32_t&         entityCount [[buffer( 2 )]],
        device const MaterialGPU*  materials [[buffer( 3 )]],
        constant CameraGPU&        camera [[buffer( 4 )]],
        constant RenderParams&     params [[buffer( 5 )]],
        texture2d<float, access::write> outTexture [[texture( 0 )]],
        uint2                      gid [[thread_position_in_grid]] )
{
        if ( gid.x >= params.width || gid.y >= params.height ) return;
        uint32_t seed = pcgHash( params.frameSeed ^ ( gid.y * 9781u + gid.x * 6271u +
1u ) );
        simd_float3 colorSum = makeFloat3( 0.0, 0.0, 0.0 );

        for ( uint32_t s = 0; s < params.samplesPerPixel; ++s )
        {
                /* jittered getRay() + rayColor(), exactly as on the CPU */
        }

        outTexture.write( float4( /* gamma-corrected average */, 1.0 ), gid );
}
```

The kernel is dispatched with `dispatchThreads:threadsPerThreadgroup:` using 8×8 threadgroups — that variant auto-clamps threadgroups that run past the image's edge, unlike the book's CUDA port, which has to compute manual grid-rounding math itself. The Transform/Shape component buffers and the material buffer all use `MTL::ResourceStorageModeShared`, meaning on Apple Silicon's unified memory architecture the same physical memory is directly visible to both the CPU (which fills it in) and the GPU (which reads it) — no explicit copy step required in either direction.

Every call to `render()` performs one untimed "warm-up" dispatch before the timed one, and surfaces *two* separate timing numbers: ordinary wall-clock time around the whole dispatch-and-wait call, and a GPU-only time read from the command buffer's own `GPUStartTime()` / `GPUEndTime()` timestamps — the latter is the headline number, since it excludes CPU-side dispatch and driver overhead that wall-clock time can't separate out.

A PACKAGING DETAIL WORTH KNOWING

`Raytracer.metal`'s `#include` of the shared header can't be compiled from an in-memory source string at runtime — Metal has no idea what file that string is supposed to be relative to, so the relative `#include` can't resolve. This project's build compiles `Raytracer.metal` to a real `.metallib` file ahead of time (via `xcrun metal` / `metallib`, wired into the CMake build), and `GPURenderer` loads that compiled library by path at startup instead of compiling shader source text on the fly.

13. The UI Layer, in Pure C++

This entire application — window, buttons, text fields, on-screen image previews, even the application menu bar — is built without a single line of Objective-C or Swift, using Apple's header-only `metal-cpp` and `metal-cpp-extensions` C++ bindings, which wrap Cocoa's Objective-C runtime message-sending underneath plain C++ classes.

Displaying a rendered image uses Metal itself rather than `NSImageView`: `App/ImageDisplayView` is a small `CAMetalLayer`-backed view with a tiny textured-quad "blit" shader (`Shaders/Blit.metal`) that samples a source texture and draws it to the screen — the same source texture is either a CPU-uploaded RGBA8 buffer or, for the GPU renderer, the compute kernel's output texture directly, with no CPU round-trip needed for the GPU path at all.

Rendering itself never runs on the main thread: each of the "Render CPU," "Render GPU," and "Compare" actions spawns a background `std::thread`, and when it finishes, marshals the resulting image and timing numbers back to the main thread via `dispatch_async(dispatch_get_main_queue(), ...)` — Grand Central Dispatch's plain C API, which needs no Objective-C either. The on-screen controls are disabled for the duration of a render, which — since settings are read on the main thread before the background thread starts — closes off any window for a second click to race the first render's in-flight state.

`ControlsPanel`'s "**Floating?**" checkbox (see §10 for the scene-side half) is a real `NSButtonTypeSwitch` button rather than the CPU-threading toggle's "push button whose title changes" trick — AppKit manages a switch button's own checked state on click, so no click handler is wired up at all; `currentSettings()` simply reads `state()` on demand when a render starts. That needed one more small addition to `NSButton.hpp`:

`setButtonType()`, `setState()`, and `state()`, following the same `sendMessage` idiom as everything else in this section.

Two more buttons, "**Save glTF**" and "**Save OBJ**", are wired the same way as the render buttons above, and — since exporting now also renders a preview image — run on a background thread the same way too, for the same reason. See §16 for the scene-export feature they trigger.

The magnifying-glass loupe

Both `ImageDisplayView`s support a synchronized loupe: hovering the mouse over either the CPU or the GPU preview zooms the *same* normalized image position in **both** previews at once, so fine per-pixel detail can be compared directly between the two renderers rather than just magnifying one image in isolation.

The zoom itself is a lens effect baked directly into `Shaders/Blit.metal`'s fragment shader: within a circular, aspect-corrected radius around a lens center, the shader samples a de-magnified region of the source texture instead of the pixel directly under the cursor, with a thin border ring drawn at the lens's edge. Because the lens is pure fragment-shader math rather than a second render pass, it costs nothing beyond the existing blit.

Mouse position reaches that shader via a local event monitor rather than the usual target/action wiring, since there's no discrete "click" to hook — the effect needs to track continuous motion:

```
NS::Event::addLocalMonitorForEventsMatchingMask( NS::EventMaskMouseMoved,
        ^NS::Event*( NS::Event* event )
        {
                /* convertPoint(event.locationInWindow, fromView: nil) into each
preview's local
                   coordinates, flip AppKit's Y-up to the texture's V-down, forward to
both lenses */
                return event;
        } );
```

That block-based API, and `NS::View::convertPoint(point, fromView)` for the window-to-view coordinate conversion, were both missing from vendored `metal-cpp-extensions` and added the same way as every other gap in this section — Objective-C blocks pass through `sendMessage` as an ordinary argument in plain C++, the same as the deallocator block `MTL::Device::newBuffer` already takes elsewhere in metal-cpp.

This feature was verified three separate ways rather than assumed correct: an offscreen shader test with a known 2px marker confirmed exact magnification with no positional drift; a coordinate-math test using production-matching window/view frames confirmed the `convertPoint` plus Y-flip math lands on the right texel; and an end-to-end test through the real scene/renderer/`Blit.metal` pipeline confirmed the lens zooms into actual, recognizable, correctly-positioned scene detail — not just a plausible-looking blur.

Filling real gaps in Apple's own bindings

Apple's `metal-cpp-extensions` turned out to be missing several pieces of AppKit this project genuinely needed — confirmed missing, not merely assumed: no button, no text field, no alert, and no `CAMetalLayer` wrapper at all (an Apple DTS engineer has confirmed on Apple's own developer forums that the AppKit coverage gap is permanent, not an oversight). Each gap was filled the same way, following the exact idiom Apple's own headers use elsewhere — a thin C++ class whose methods call straight through to Objective-C's message-sending runtime function, resolved by selector name:

```
void NS::Control::setEnabled( bool enabled )
{
        Object::sendMessage< void >( this, sel_registerName( "setEnabled:" ), enabled
);
}
```

One of these gaps was worth catching rather than working around silently: Apple's own vendored `NS::View::init()` sent `initWithFrame:` to the NSView *class object itself* instead of to an allocated instance — a genuine bug, confirmed by reproducing the resulting crash directly before fixing it, not merely inferred from reading the header.

14. Testing and the Parity Investigation

`RayTracerBenchTests` is a plain command-line C++ executable (no XCTest, no Objective-C), using a small dependency-free self-registering test framework written for this project rather than vendoring an external one. It covers two things:

- **Unit tests for `hitSphere()` and `hitPyramid()`** — a dead-on hit, a clean miss, the `tMax` boundary, a ray originating *inside* a sphere (which must report a back-face hit, per Part I §3), picking the closer of two overlapping spheres, and — for the pyramid — hitting the apex, hitting a sloped side face off-axis, hitting the base from below, and

confirming `hitEntity()` actually dispatches to each shape's own intersection function rather than always taking one path.

- **`EntityMeshTests.cpp`** — the scene-export mesh builder (§16) is checked the same way the intersection code is: every triangle's winding is confirmed to face outward via its cross-product normal, vertex/triangle counts are checked exactly for the pyramid's faceted mesh, and a rotated pyramid's apex is confirmed to land where its orientation basis actually points.
- **Deterministic CPU/GPU parity** — the same fixed seed, rendered by both renderers, diffed pixel-by-pixel.

The parity check is worth explaining in a bit of detail, because the naive expectation — "same algorithm, same seed, should match closely" — turned out to be only approximately true, and understanding *why* is itself a useful piece of ray-tracer theory. At a low sample count (16 samples per pixel), roughly 9% of color channels differed by more than a couple of 255ths, with some differences as large as 61. Investigated rather than dismissed, the pattern was unambiguous: every pixel that never hit any geometry (pure sky) matched *exactly*, with zero difference, while mismatches appeared only on pixels where a ray actually struck a surface and had to scatter — and raising the sample count from 16 to 200 cut both the mismatch rate and the worst-case difference roughly in half.

That combination of evidence points to one specific cause: a sub-ULP (unit-in-the-last-place) floating-point difference between the CPU's and the GPU's `sqrt` / `pow` implementations can, on rare occasions, flip a decision that depends on comparing two nearly-equal floats — which root of the quadratic is closer, whether Schlick's approximation exceeds a random draw, whether total internal reflection applies. Once such a decision flips, the rest of that one ray's bounce path is a *different path through the scene*, not a slightly different color — which is exactly why individual differences can be large even though the underlying computation is correct on both sides. This is a well-known property of chaotic systems in general and Monte Carlo path tracers in particular: a real bug would not shrink as more samples are averaged together, but this kind of tiny-input-sensitivity does, because it only ever affects a small, roughly constant fraction of individual sample rays. The committed test asserts on the overall mismatch rate at a high sample count, where that averaging has mostly settled, rather than on a strict per-pixel bound at a low one.

15. The Build System

The Xcode project is not hand-authored — it's generated from a single `CMakeLists.txt` via CMake's Xcode generator (`cmake -G Xcode`), then copied over the tracked `.xcodeproj`. Three targets fall out of that one file:

Target	Contents
`RayTracerCore`	Static library: `Core/Scene.cpp` + `CPU/CPURenderer.cpp` — the framework-free half of the project.
`RayTracerBench`	The app itself: `App/`, `GPU/GPURenderer.cpp`, linked against Cocoa/Metal/MetalKit/QuartzCore/Foundation.
`RayTracerBenchTests`	The headless test executable — only Metal + Foundation, no AppKit at all.

A custom build step compiles `Raytracer.metal` to a `.metallib` ahead of time (see the note in §12), and the same compiled library is shared by both the app and the test target, so the deterministic-parity test exercises the identical GPU code path the real application uses.

16. Scene Export: glTF and OBJ+MTL

The "Save glTF"/"Save OBJ" buttons (§13) export the *current scene's geometry* as glTF or OBJ+MTL, and — since neither of those is a rendered image, and a 3D file with no quick way to see what it contains is awkward to work with — also write a same-named `.png` preview alongside it. Both the geometry writers and the mesh builder live in a new `Export/` module, kept framework-free (no Metal, no AppKit) and built into the same `RayTracerCore` static library as `Core/`, since they never need either; the preview-image writer is the one piece of `Export/` that isn't, for reasons its own subsection below explains.

`buildEntityMesh()`: A Third ECS System

§9 introduced `hitEntity()` as a "system" dispatching on `ShapeGPU::type` to test ray intersections, and `scatter()` as one dispatching on `MaterialGPU::type` to compute shading. `Export/EntityMesh.hpp`'s `buildEntityMesh()` is a third: the same

Transform+Shape components, the same tagged dispatch, but this time building a triangle mesh instead of testing a ray:

Export/EntityMesh.hpp (abridged)

```
struct MeshData
{
        std::vector<simd_float3> positions;
        std::vector<simd_float3> normals;
        std::vector<uint32_t>    indices; // triangle list
};

MeshData buildEntityMesh( TransformGPU transform, ShapeGPU shape );
```

Neither glTF nor OBJ has a native sphere primitive, so every sphere (including the giant ground sphere) is tessellated into an 8×12 latitude/longitude grid with smooth, per-vertex outward normals — the same shading a real sphere would have. One subtlety: every longitude column collapses to a single point at each pole, so one of the two triangles in each pole-row quad is exactly zero-area; rather than emit that degenerate triangle and let it sit in the file as useless data, the mesh builder detects and skips it, emitting only the real triangle in that quad.

Pyramids need no tessellation at all — they're already flat-faced. `buildEntityMesh()` builds them from the exact same 5 local vertices (apex plus 4 base corners) that §3/§9's `hitPyramidLocal()` derives its half-space planes from, transformed to world space through the same Transform basis `hitPyramid()` uses. Each of the 6 faces (4 sides + a 2-triangle base) gets its own 3 unique vertices rather than sharing vertices with its neighbors, so each face has one flat per-triangle normal instead of an averaged one — correct for a shape whose edges are meant to look sharp, not smoothed.

VERIFIED, NOT ASSUMED — AND IT CAUGHT A REAL BUG

Winding order (which direction around a triangle's 3 vertices makes its computed normal point outward rather than inward) is easy to get backwards without noticing, since it doesn't show up as a compile error. A first attempt at the sphere mesh's winding looked plausible by inspection but was actually wrong: a standalone check computing every triangle's cross-product normal and confirming it pointed away from the sphere's center found 174 of 192 triangles facing inward. Fixed, and kept as a permanent test (`EntityMeshTests.cpp` , §14) rather than trusted by eye a second time.

Two Formats, One Set of Materials

`Export/SceneExporter.hpp` writes either format from the same `SceneDescription`. A glTF export is a single self-contained `.gltf` file — the binary mesh data (positions/normals/indices for every entity) is embedded directly in the JSON as a base64 `data:` URI buffer, rather than a separate `.bin` file, so opening the export only ever means opening one file. An OBJ export is the traditional pair: a `.obj` (geometry, referencing materials by name) alongside a companion `.mtl` (the material definitions themselves).

Materials map onto each format's native model: glTF's PBR metallic-roughness parameters (`baseColorFactor`, `metallicFactor`, `roughnessFactor`) for lambertian/metal, and OBJ/MTL's classic `Kd` / `Ks` / `Ns` for the same. Metal's `fuzz` parameter (§4) maps directly onto roughness in both formats, since that's exactly the role it already plays in `scatter()`. Neither format's core specification has a real glass/transmission model, though — `MAT_DIELECTRIC` is approximated the same conceptual way in both: a clear, glossy, partly-transparent surface (`alphaMode: BLEND` with low alpha in glTF; a low `d`/dissolve value and `illum 4` — "transparency: glass on, reflection: ray trace on" — in MTL). This is documented as an approximation, not presented as physically equivalent to what the raytraced image actually shows; glTF's `KHR_materials_transmission` extension was deliberately left unused so the export stays a plain, universally-loadable file rather than one that only renders correctly in viewers supporting that extension.

A Preview Image, Same Name, Same Place

A 3D file is opaque at a glance — there's no way to tell what a `.gltf` actually contains without loading it into a viewer. So a successful export doesn't stop at the geometry: the same scene is also rendered at whatever settings are currently dialed in (exactly the image "Render CPU" would produce for that seed) and saved as a `.png` with the identical stem, in the identical `SavedScenes/` directory — a quick-look thumbnail sitting right next to the file it describes.

`Export/ImageWriter.hpp/.cpp` writes that PNG using CoreGraphics/ImageIO's plain C API (`CGImageCreate`, `CGImageDestinationCreateWithURL`, `CGImageDestinationAddImage` / `Finalize`) — real system frameworks, so this needs no vendored image library and, like every other AppKit/Metal/QuartzCore integration in this project, no Objective-C. It's the one file in `Export/` that isn't part of the framework-free `RayTracerCore` library: `EntityMesh` and `SceneExporter` need nothing beyond the C++ standard library, but writing an actual PNG needs the CoreGraphics and ImageIO

frameworks, so `ImageWriter.cpp` is linked directly into the `RayTracerBench` app target instead.

> A DESIGN DECISION THIS FEATURE BROKE, AND HOW IT WAS FIXED
>
> §13 was written when the Save buttons only wrote geometry — cheap enough that `AppDelegate::saveScene()` ran synchronously on the main thread, unlike the render actions, which always use a background thread. Adding a full CPU render for the preview image invalidated that: a render at high sample counts or image widths is exactly the kind of main-thread-blocking work this project goes out of its way to avoid elsewhere (see §13's heartbeat-timer verification). `saveScene()` was rewritten to spawn a background `std::thread` — building the scene, exporting it, rendering the preview, and writing the PNG all happen off the main thread, with the result alert marshaled back via `dispatch_async(dispatch_get_main_queue(), ...)`, exactly like `startCPURender()` / `startGPURender()` / `startCompare()` already do.

Where the File Goes, and What It's Named

Exports are written to a `SavedScenes/` subdirectory *next to the running executable* — not the current working directory, which depends on how the app happened to be launched (Xcode's "Run," a Finder double-click, and a terminal invocation from three different directories would otherwise produce three different save locations for the exact same click). The real executable path is resolved via `_NSGetExecutablePath` and canonicalized, rather than trusting `argv[0]` (which can be a relative path or a symlink) or `getcwd()` (which has nothing to do with where the binary lives). The preview image needs to land in that exact same directory, so `SceneExporter.hpp` exposes `ensureSavedScenesDirectoryPath()` publicly — the same resolution-plus-creation logic the geometry writers already use internally, rather than a second copy of it in `AppDelegate`.

The filename itself is built to make the exported scene's parameters identifiable at a glance: the seed and image width (both of which determine the entity layout and camera, per §10) and the Floating? checkbox's state, plus a timestamp so exporting the same settings twice doesn't silently overwrite the earlier file. Samples-per-pixel and max-depth are deliberately left out — they govern how a scene is *rendered*, and have no bearing on the geometry an export actually contains.

Attribution

This project is based on the algorithm and structure of two reference works — structural precedent only, no source copied:

- "**Ray Tracing in One Weekend**" by Peter Shirley, Trevor David Black, and Steve Hollasch (*raytracing.github.io*). CC0 (public domain) — no attribution legally required, credited here as a courtesy.
- **raytracinginoneweekendincuda**, a CUDA port of the above by Roger Allen (*github.com/rogerallen/raytracinginoneweekendincuda*). Public domain — likewise credited as a courtesy, not a requirement.
